The Value of a Wrong Plan

Iterated Greedy Sequencing for Mean Total Weighted Tardiness in a Stochastic Flow Shop

Muting (Don) Ma and Amirhossein Ranjbar

OM 522 team project, option 6

November 9, 2026

github.com/mutingma123/OM522Project

The talk in six parts

1. The question (1:25)
2. Model and methods (3:40)
3. Experiment design (0:40)
4. Results (3:10)
5. Recommendation and limits (0:55)
6. Walk-throughs (1:15)

Minutes are the planned share of the video.

A planner must fix one job order before the times are known

The decision

- Eight stages in a fixed order; jobs cannot pass, so one sequence holds at every stage.
- Each job has a release date, a due date, and a weight.
- Primary measure: total weighted tardiness.
- Secondary measure: total waiting between stages under earliest-start execution, in job-minutes.

The rule to beat

- Earliest due date (EDD) is among the most common rules for tardiness [1].
- It sorts by due date alone: no weights, no releases, no later-stage times.
- We hold its list frozen, as the planner must.

A job that is slow at a later stage holds up every job behind it.

Is a plan optimized on wrong times still worth having?

Known on exact data

- Insertion heuristics beat dispatching rules for tardiness [1].
- Iterated greedy was the best method tested for weighted tardiness, on flow shops with setups [2].

Known with random times

- Studies concentrate on the expected makespan [4].
- Methods built on exact or mean times stay close to the best stochastic methods [5, 6, 7].
- Tardiness is named as open [5, 4].

Warning from schedule execution

Optimized schedules lose their advantage beyond a level of variability that depends on the system [3].

The studies we reviewed do not test weighted tardiness with release dates when a searched sequence is held fixed.

One sequence fixes every start time

$$S_{\pi(i),k} = \max\{C_{\pi(i),k-1},\, C_{\pi(i-1),k}\}, \qquad C_{\pi(i),k} = S_{\pi(i),k} + p_{\pi(i),k}$$

$$Z(\pi) = \sum_{j} w_j \max\{0,\, C_{jm} - d_j\}, \qquad W(\pi) = \sum_{j} \sum_{k \geq 2} \left(S_{jk} - C_{j,k-1}\right)$$

Assumptions

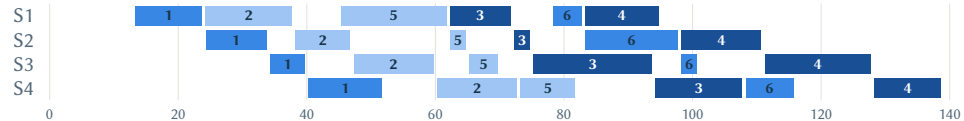
- Every stage is free at time zero; buffers are unlimited.
- The sequence is fixed once and never revised.
- With equal tardiness, the planner prefers less waiting.

Certified optimum

A positional mixed-integer program defines the optimum. HiGHS [8] certifies it on the 6- and 15-job instances, as a benchmark only.

Rule to beat: earliest due date gives 281 on the hand-checkable case

Sort jobs by due date (ties by job identifier) and use that order at every stage; each operation starts at the later of the job's previous completion (or its release) and the stage's previous completion.



Bars carry the job number; shading is the job's weight: 1 2 4. Minutes on the axis.

J001 starts S1 at its release 13; J002, released at 8, waits for S1 until 24; S1 then sits idle until J005 is released at 45.

Weighted tardiness 281 and waiting 13: the decision is identical to the worked example.

Separate code confirms each decision from the input files

What the checker confirms

- Each job once in the sequence; each job-stage pair once in the schedule.
- Each duration equals the processing time within 10^{-8}.
- Release at S1, then stage order, for every job.
- One job at a time per stage, in the sequence's order.

What it reported

- Worked example: feasible; every recomputed measure matches the worked example's measures.
- Fresh copy of the 45-job development instance under EDD: feasible, weighted tardiness 2,371.
- Edge case: rejected with a message naming the file, column, and line of the zero processing time; no decision is written; exit status 1.

It recomputes every measure and imports nothing from the solver, so a method's timing error cannot repeat here.

Researched method: iterated greedy

Source

Ruiz and Stützle's IG_RS_LS for flow shops with setup times and weighted tardiness [2], built on their makespan version [9]; we set every setup time to zero.

Why it should beat EDD

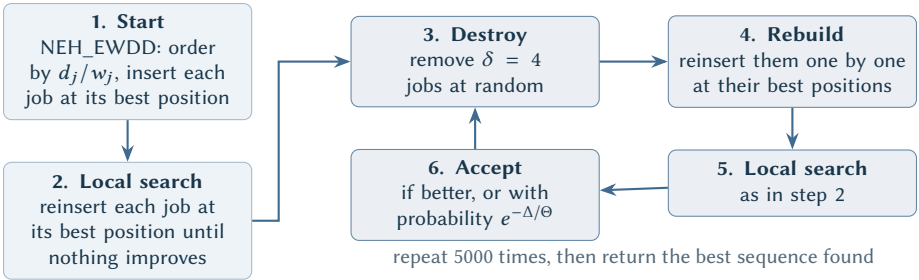
It compares sequences by their actual weighted tardiness on all eight stages, so a heavy job can move ahead of a light one.

Evidence in the sources

- Its weighted start, NEH_EWDD, beat plain NEH on 470 of 480 instances [2].
- It had the lowest average deviation from the best known solutions in all four instance sets [2].

Neither source has release dates; our timing includes them in every evaluation.

How it builds, improves, and stops



- Solution: a permutation of the jobs.
- Compare: weighted tardiness, then waiting.
- $\delta = 4$, $\lambda = 0.5$, $\Theta = \lambda \sum_{j,k} p_{jk}/(10\,n\,m)$ [2].
- Stop: 5000 iterations, seed 522; a 300 s cap is only a safety limit.
- Start from the better of NEH_EWDD and EDD, so the result is never worse than the rule.

Four instances, one checker, 180 paired scenarios

Instances and settings

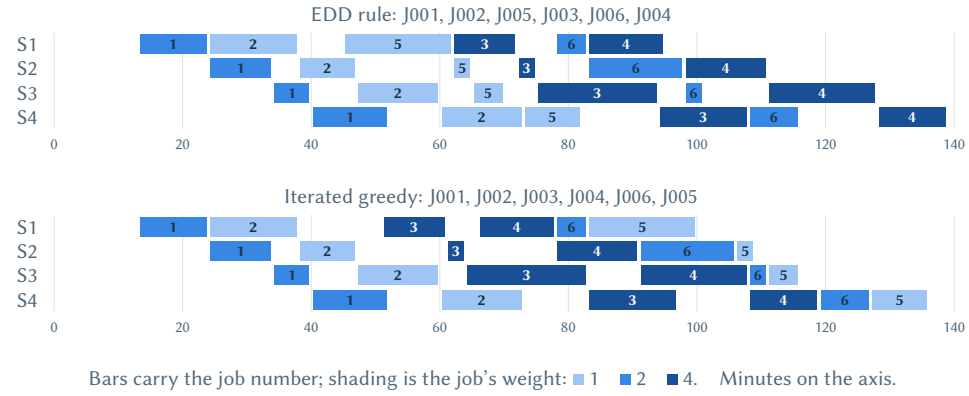
Instance	Jobs	Stages
Hand-checkable	6	4
Small development	15	8
Medium development	45	8
Large development	120	8

Iterated greedy: 5000 iterations, seed 522. The independent checker computes every reported value.

Stress test on the 45-job instance

- Each planned time is multiplied by a lognormal factor with mean one and coefficient of variation ν.
- 30 saved scenarios at $\nu = 0.2$, then 30 new ones at each of $\nu = 0.1$, 0.2, 0.3, 0.5, and 0.8.
- Both sequences stay fixed; each operation starts as early as the realized times allow.

On the hand-checkable case it reaches the optimum: 233 against 281



Moving the weight-1 job J005 last lets the weight-4 jobs finish earlier: J003 at 97 instead of 108 and J004 at 119 instead of 139.

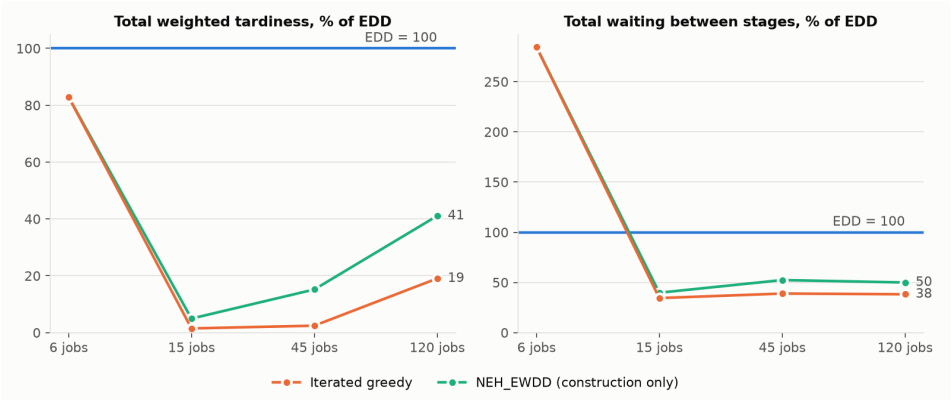
All 720 sequences confirm 233 is optimal; waiting rises from 13 to 37 job-minutes.

On exact data, iterated greedy (IG) is lower than EDD on all four instances

		Weighted tardiness		Waiting (job-min)		Makespan		On-time fraction	
Instance	Jobs	EDD	IG	EDD	IG	EDD	IG	EDD	IG
Hand-checkable	6	281	233	13	37	139	136	0.000	0.000
Small	15	1,417	21	732	252	417	357	0.333	0.867
Medium	45	2,371	58	2,040	794	781	751	0.244	0.911
Large	120	23,015	4,383	7,863	3,005	1,812	1,804	0.233	0.642

- All eight decisions pass the checker. Iterated greedy (IG) takes 0.21, 0.52, 3.22, and 37.2 seconds; EDD takes under a millisecond.
- HiGHS proves 233 and 21 optimal on the 6- and 15-job instances, and iterated greedy reaches both. At 15 jobs the solver's one optimal order has 323 job-minutes of waiting, 71 more than iterated greedy's.
- At 120 jobs, 5000 iterations have not converged: five other seeds reached 4,159 to 4,306.

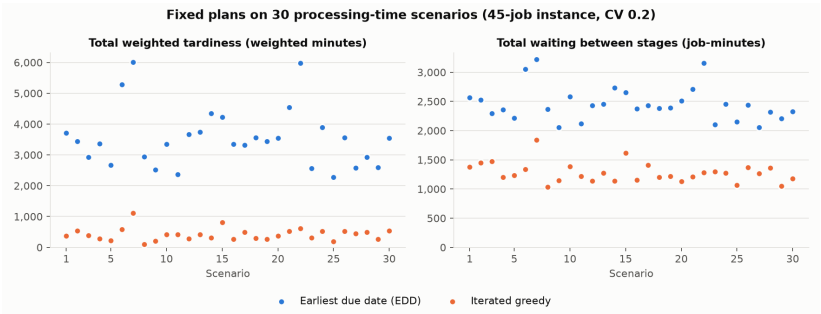
The gain relative to EDD is largest at 15 and 45 jobs



At 120 jobs iterated greedy keeps 19 percent of the EDD tardiness; five other seeds there reached 4,159 to 4,306, so the search has not converged.

At $\nu = 0.2$ the iterated greedy sequence wins all 30 scenarios

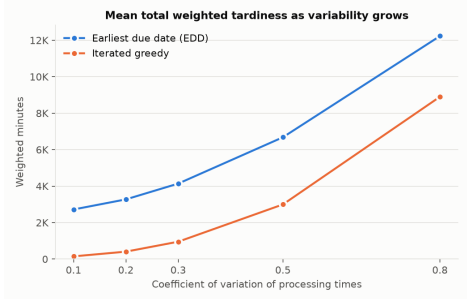
	Weighted tardiness			Waiting (job-minutes)		
Sequence	Planned	Mean	90th pct.	Planned	Mean	90th pct.
Earliest due date	2,371.0	3,541.1	4,616.6	2,040.0	2,454.5	2,768.6
Iterated greedy	58.0	420.6	592.2	794.0	1,276.9	1,454.9



Mean reduction: 3,120.5 weighted minutes (standard error 150.1). No hard constraint is broken.

As variability grows, the percentage falls and the mean gain holds

	Mean tardiness		Reduction		
ν	EDD	IG	Mean (s.e.)	10th pct.	%
0.1	2,720.1	148.1	2,572.0 (65.9)	2,048.3	94.6
0.2	3,273.5	407.2	2,866.4 (121.3)	2,055.3	87.6
0.3	4,139.8	952.5	3,187.3 (203.8)	1,781.1	77.0
0.5	6,680.4	2,996.7	3,683.7 (223.8)	2,473.2	55.1
0.8	12,230.5	8,901.6	3,329.0 (342.7)	851.5	27.2



EDD wins 1 of the 180 paired scenarios, at $\nu = 0.8$, by 870.4 weighted minutes.

The percentage falls at every step. The mean gain stays between 2,572.0 and 3,683.7 weighted minutes.

Why: more on-time jobs to lose, each at lower risk

At $\nu = 0.2$, 30 saved scenarios	EDD	Iterated greedy
Late jobs in the plan	34	4
Mean late jobs across scenarios	36.1	11.5
On-time jobs in the plan	11	41
Median planned slack of on-time jobs (minutes)	28	31
Median simulated completion standard deviation (minutes)	9.9	8.3
Mean late probability of an on-time job	27.9%	18.5%
On-time jobs that turn late per scenario	3.1	7.6

Simulated completion times include the delays passed on by the jobs ahead.

Recommendation: use the iterated greedy sequence on this line

Expected benefit at $\nu = 0.2$ on the 45-job instance

Mean weighted tardiness falls by 3,120.5 (standard error 150.1) and, in a second independent set, by 2,866.4 (standard error 121.3). Late jobs per period: 11.5 against 36.1. Mean waiting is 48.0 percent lower.

Trade-off we accept

The sequence cannot be traced by hand, depends on a seed, and takes seconds to compute: 3.22 at 45 jobs and 37.2 at 120.

Condition that would change it

At $\nu = 0.8$ the mean gain remains 3,329.0, but the 10th percentile of the per-period gain falls to 7.0 percent of the EDD mean and EDD wins 1 of 30 scenarios. Measure the line's variability first.

Limits of the evidence and what comes next

What the study does not show

- One generator; the stress test covers one 45-job instance.
- Independent noise: no slow shift or degraded machine.
- EDD is a frozen list, not online dispatching.
- NEH_EWDD and stochastic methods were not run in the scenarios.
- Optimality is proven only up to 15 jobs.

Next tests

- Score sequences over sampled scenarios.
- Add correlated delays by stage or shift.
- Re-sequence at each release.

Walk-through (Don): the mixed-integer benchmark

Drafted by Claude Code

What it does

Builds the positional program and certifies the optimum of the 6- and 15-job instances with HiGHS.

What was wrong in the agent's version

It set the objective with a call that also solves, then solved again. The solver reported 81 seconds, but the command took more than three minutes.

How we checked it

We compared the reported time with the wall clock and removed the second solve. A test requires the proven optimum of 233, and the checker confirms the solver's order.

Walk-through (Amir): which on-time jobs are at risk

Drafted by Claude Code

What it does

Measures how exposed the on-time jobs of each sequence are when processing times vary.

What was wrong in the agent's version

It compared slack with the variation of a job's own processing times. That misses delays passed on by the jobs ahead.

How we checked it

We replaced it with simulated completion times over the 30 saved scenarios. Jobs with slack below that variation became 6 of 41 and 4 of 11, where the first version showed 3 in each sequence.

What we now know

- On exact data, iterated greedy cuts weighted tardiness by 81.0 to 98.5 percent against EDD on the development instances and reaches the proven optimum on the two smallest instances.
- At $\nu = 0.2$, its fixed sequence wins all 30 scenarios, with an 88.1 percent lower mean.
- As variability grows, the percentage falls to 27.2 at $\nu = 0.8$, while the mean gain stays between 2,572.0 and 3,683.7 weighted minutes.

A plan optimized on wrong times kept its value on this line at every level of variability we tested.

Sources I

[1] Eva Vallada, Rubén Ruiz, and Gerardo Minella.
Minimising total tardiness in the m-machine flowshop problem: A review and evaluation of heuristics and metaheuristics.
Computers & Operations Research, 35(4):1350-1373, 2008.
doi:10.1016/j.cor.2006.08.016.

[2] Rubén Ruiz and Thomas Stützle.
An Iterated Greedy heuristic for the sequence dependent setup times flowshop problem with makespan and weighted tardiness objectives.
European Journal of Operational Research, 187(3):1143-1159, 2008.
doi:10.1016/j.ejor.2006.07.029.

[3] Haldun Aytug, Mark A. Lawley, Kenneth McKay, Shantha Mohan, and Reha Uzsoy.
Executing production schedules in the face of uncertainties: A review and some future directions.
European Journal of Operational Research, 161:86-110, 2005.
doi:10.1016/j.ejor.2003.08.027.

[4] Eliana María González-Neira, Jairo R. Montoya-Torres, and David Barrera.
Flow-shop scheduling problem under uncertainties: Review and trends.
International Journal of Industrial Engineering Computations, 8:399-426, 2017.
doi:10.5267/j.ijiec.2017.2.001.

Sources II

[5] Kenneth R. Baker and Dominik Altheimer.
Heuristic solution methods for the stochastic flow shop problem.
European Journal of Operational Research, 216:172-177, 2012.
doi:10.1016/j.ejor.2011.07.021.

[6] Jose M. Framinan and Paz Perez-Gonzalez.
On heuristic solutions for the stochastic flowshop scheduling problem.
European Journal of Operational Research, 246:413-420, 2015.
doi:10.1016/j.ejor.2015.05.006.

[7] Angel A. Juan, Barry B. Barrios, Eva Vallada, Daniel Riera, and Josep Jorba.
A simheuristic algorithm for solving the permutation flow shop problem with stochastic processing times.
Simulation Modelling Practice and Theory, 46:101-117, 2014.
doi:10.1016/j.simpat.2014.02.005.

[8] HiGHS development team.
HiGHS: High performance software for linear optimization, version 1.15.1, 2026.
URL: https://highs.dev.

[9] Rubén Ruiz and Thomas Stützle.
A simple and effective iterated greedy algorithm for the permutation flowshop scheduling problem.
European Journal of Operational Research, 177(3):2033-2049, 2007.
doi:10.1016/j.ejor.2005.12.009.